Taller 1 Corte 2 Esteganografía

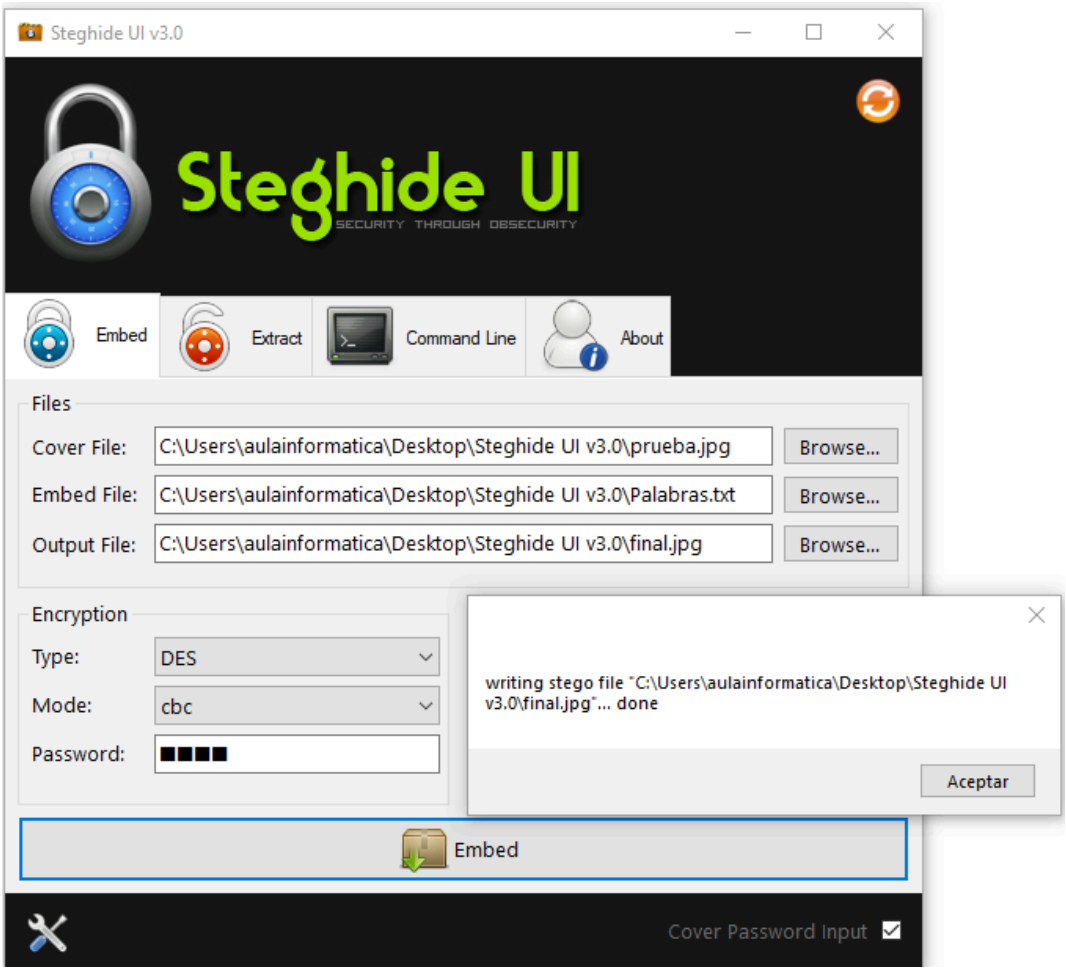
Jesús David Barrios Santamaría

Diego Alejandro Mogollon

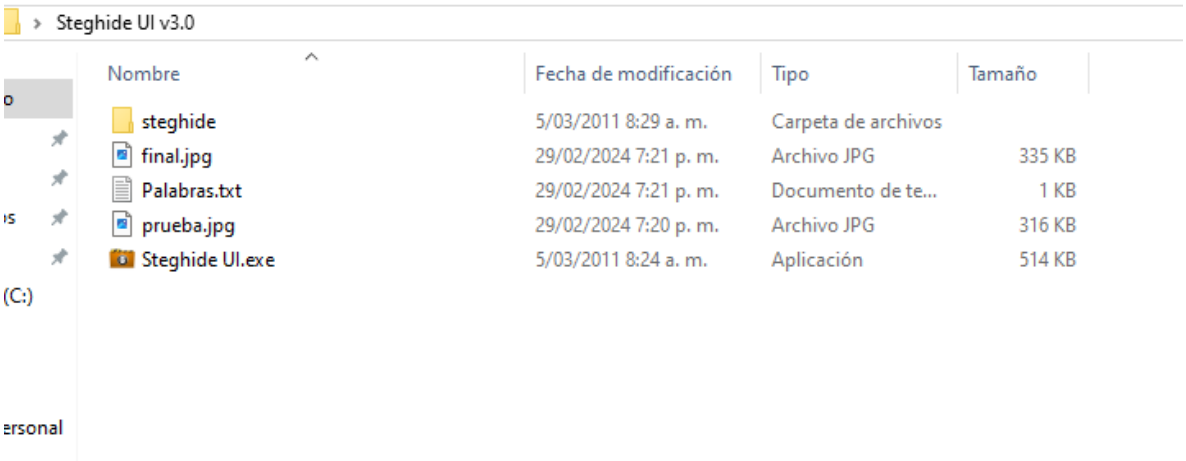
Universidad el Bosque

2024

Seguridad de la información



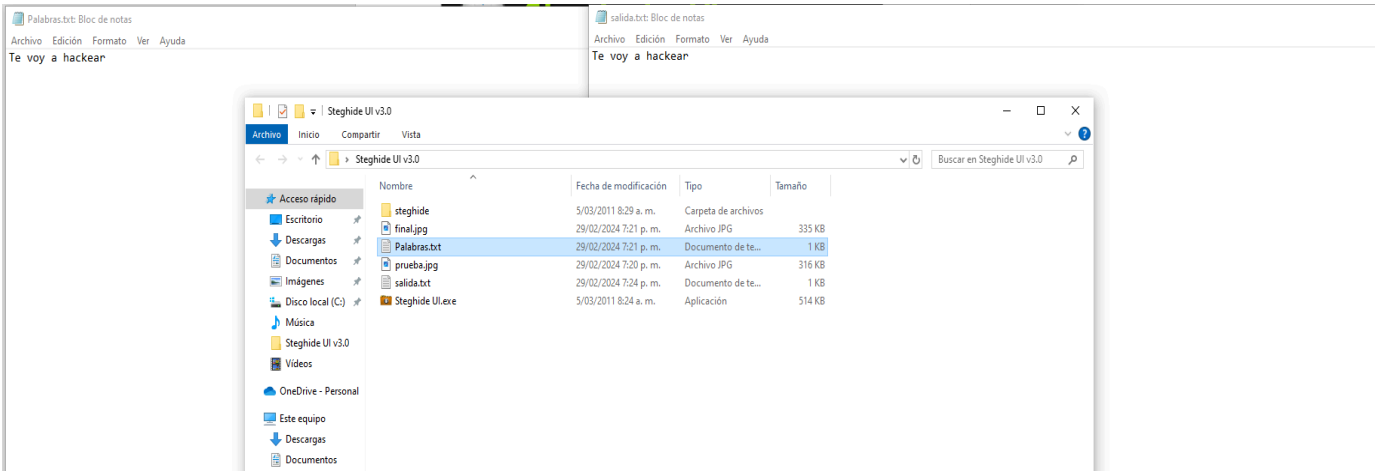
Se adiciona la imagen, junto con el txt que contiene el mensaje oculto, y por último se le asigna un nombre a la imagen que se le adicionará el mensaje oculto, en la configuración se cambia Type por DES y se asigna una clave obligatoria.



Se verifica en la ruta que se haya creado la imagen con el nombre asignado.



Luego de ello, en la parte de extract se selecciona la imagen con el mensaje oculto, y se asigna un nombre con formato txt, y la clave asignada inicialmente.



Se confirma que se guardó el archivo extraído y tiene el mismo mensaje oculto en la imagen.

¿Qué tipo de archivo se recomienda utilizar como archivo inicial en este ejercicio?

Se recomienda utilizar imágenes en formato jpg, ya que es uno de los formatos que exige el aplicativo para poder ocultar el mensaje.

¿Por qué se recomienda utilizar una contraseña para proteger el mensaje oculto?

Se recomienda usar contraseña para aumentar la seguridad del mensaje oculto en si, y porque el aplicativo la pide por obligación con longitud mínima de 4 caracteres.

¿Qué significa el término "embedding" en el contexto de la esteganografía?

"Embedding" en esteganografía significa ocultar información dentro de un archivo digital de manera que sea invisible para quienes no conocen el método utilizado, pero aun así pueda ser recuperada por aquellos que sí lo conocen.

¿Qué es lo que se busca al verificar ambas imágenes (la imagen original y la imagen con el mensaje oculto)?

Al verificar tanto la imagen original como la imagen con el mensaje oculto, el objetivo principal es determinar si hay alguna diferencia perceptible entre ellas. Si se ha realizado correctamente el proceso de esteganografía, la imagen con el mensaje oculto debería ser prácticamente idéntica a la original en apariencia visual.

Indique si ve alguna diferencia en ambas imágenes, la original y la que tiene el SMS oculto.

Ante el ojo humano, no hay ninguna diferencia entre ambas imágenes.

¿Qué herramientas o verificaciones puedo utilizar para validar si la imagen este ocultando alguna información?

Análisis visual: Inspeccionar la imagen cuidadosamente visualmente para buscar anomalías, patrones extraños o artefactos que puedan indicar la presencia de datos ocultos.

Análisis de metadatos: Verificar los metadatos de la imagen para buscar información adicional o signos de manipulación.

Extracción de datos: Extraer cualquier información oculta utilizando herramientas de esteganografía. Si se puede recuperar un mensaje oculto, entonces se confirma la presencia de esteganografía.

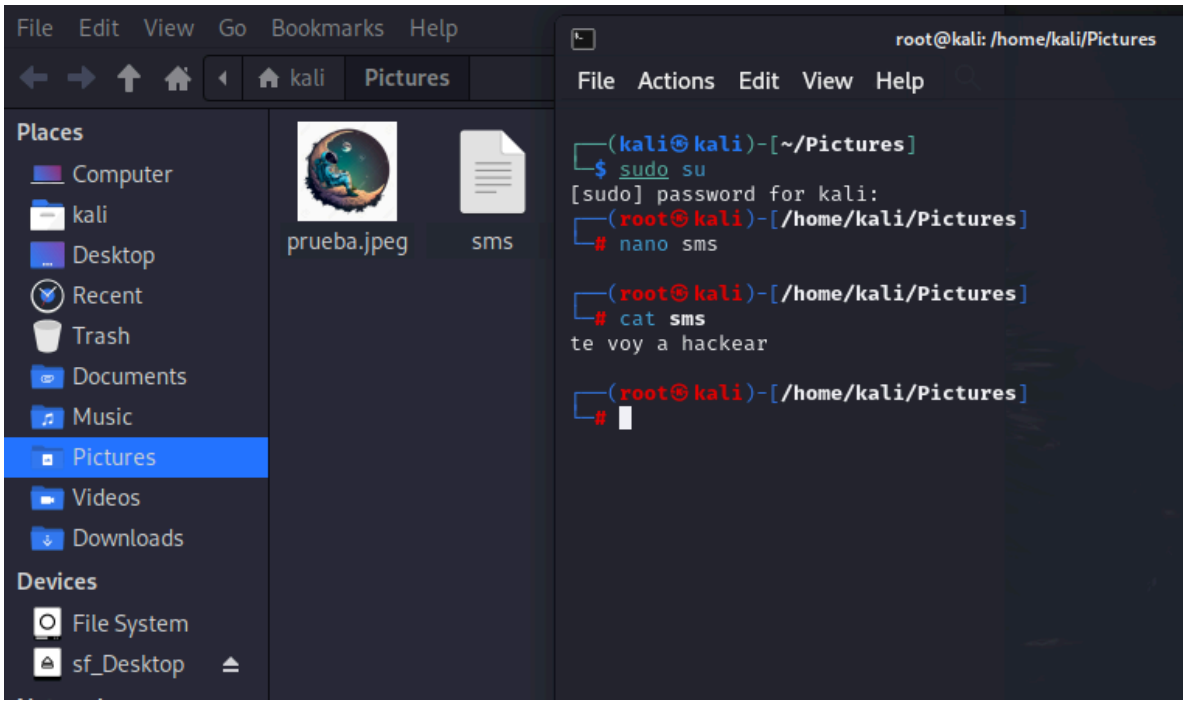
Parte 2 Kali-Linux

```
┌──(kali㉿kali)-[~]
└─$ sudo su
[sudo] password for kali:
┌──(root㉿kali)-[/home/kali]
└─# sudo apt-get update
Get:1 https://mirror.cedia.org.ec/kali kali-rolling InRelease [41.5 kB]
Get:2 https://mirror.cedia.org.ec/kali kali-rolling/main amd64 Packages [19.9
 MB]
Get:3 https://mirror.cedia.org.ec/kali kali-rolling/main amd64 Contents (deb)
 [47.3 MB]
Get:4 https://mirror.cedia.org.ec/kali kali-rolling/contrib amd64 Packages [1
21 kB]
Get:5 https://mirror.cedia.org.ec/kali kali-rolling/contrib amd64 Contents (d
eb) [261 kB]
Get:6 https://mirror.cedia.org.ec/kali kali-rolling/non-free amd64 Packages [
194 kB]
Get:7 https://mirror.cedia.org.ec/kali kali-rolling/non-free amd64 Contents (
deb) [885 kB]
Get:8 https://mirror.cedia.org.ec/kali kali-rolling/non-free-firmware amd64 P
ackages [33.0 kB]
Get:9 https://mirror.cedia.org.ec/kali kali-rolling/non-free-firmware amd64 C
ontents (deb) [16.8 kB]
Fetched 68.8 MB in 50s (1,388 kB/s)
```

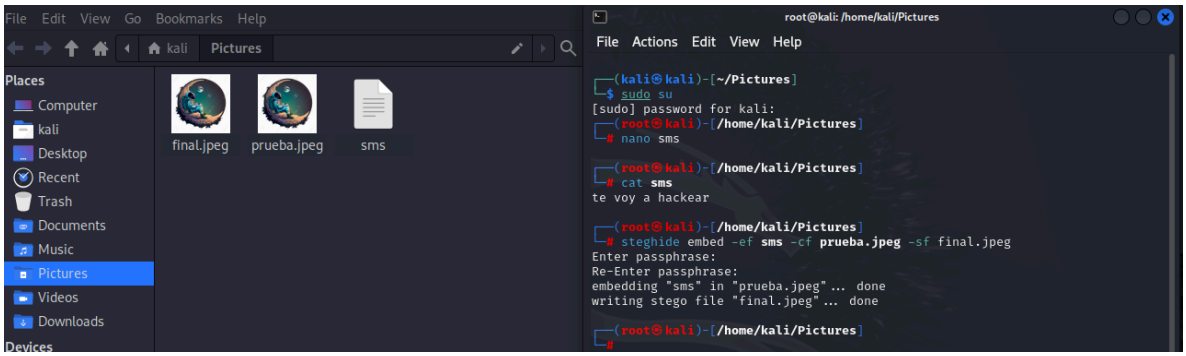
Se envía el comando sudo apt-get update y se actualiza principalmente todo.

```
┌──(root㉿kali)-[/home/kali]
└─# sudo apt-get install steghide
Reading package lists ... Done
Building dependency tree ... Done
Reading state information ... Done
The following additional packages will be installed:
  libmcrypt4 libmhash2
Suggested packages:
  libmcrypt-dev mcrypt
The following NEW packages will be installed:
  libmcrypt4 libmhash2 steghide
0 upgraded, 3 newly installed, 0 to remove and 267 not upgraded.
Need to get 309 kB of archives.
After this operation, 905 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://kali.download/kali kali-rolling/main amd64 libmcrypt4 amd64 2.5.
8-7 [72.6 kB]
Get:2 http://http.kali.org/kali kali-rolling/main amd64 libmhash2 amd64 0.9.9
.9-9+b1 [92.4 kB]
Get:3 http://kali.download/kali kali-rolling/main amd64 steghide amd64 0.5.1-
15 [144 kB]
Fetched 309 kB in 16s (19.1 kB/s)
Selecting previously unselected package libmcrypt4.
```

Luego de ello se instala el steghide.



Se guarda la imagen inicial, y se crea un archivo con el mensaje oculto.



Se evidencia que el comando lo que hace es enviar inicialmente el **embedding** con el archivo creado del mensaje oculto llamado sms, luego de ello busca la imagen inicial y la guarda con otro nombre donde estará el mensaje oculto, al final pide la contraseña de guardado.

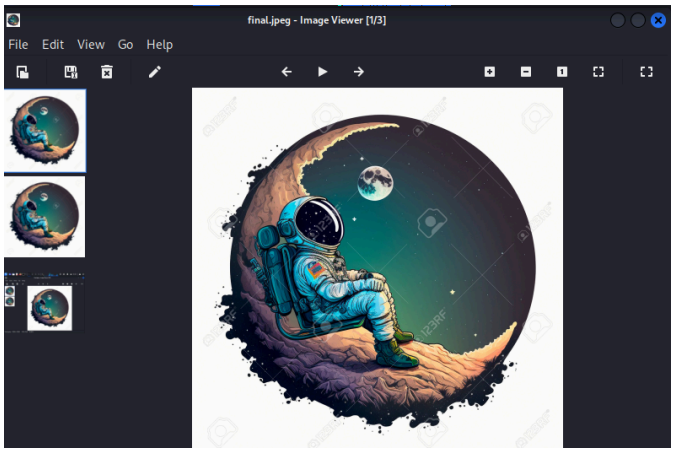


Imagen final-

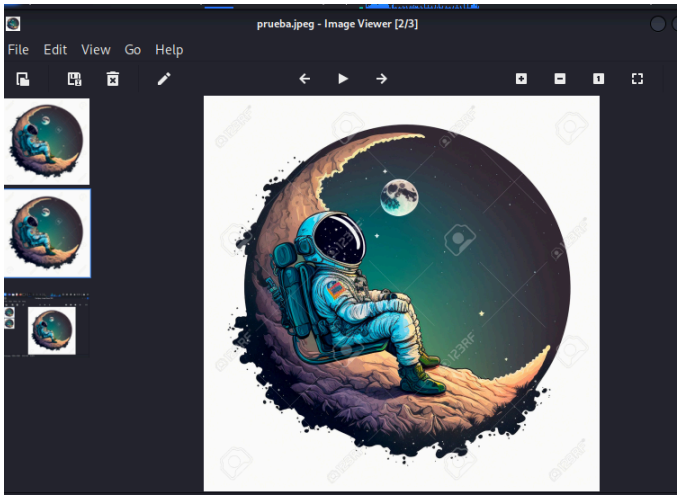
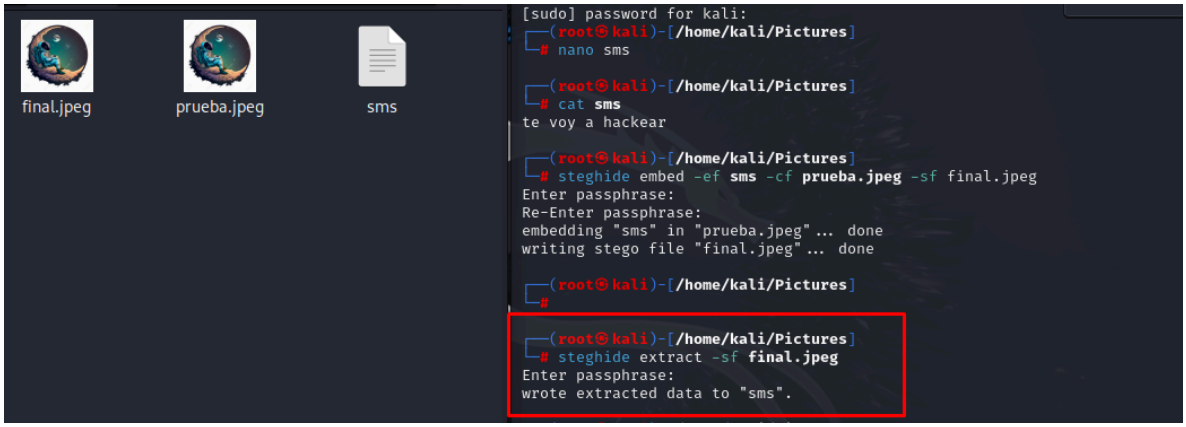


Imagen inicial.

Al ojo no es posible identificar ninguna diferencia por más que se van ambas al tiempo y detalladamente.



Se evidencia que se extrajo correctamente el archivo con el mensaje oculto.

```
┌──(root㉿kali)-[/home/kali/Pictures]
└─# nano srvestegano.py

┌──(root㉿kali)-[/home/kali/Pictures]
└─# cat srvestegano.py
#!/bin/bash
amarillo_luminoso='\033[1;33m'
sin_color='\033[0m'
archivos=$(ls)
contador=0
mkdir imagenesconsms
for variable in $archivos
do
tipo=$(file -bi "$variable")
if [[ $tipo == image/* ]]
then
contador=$((contador+1))
steghide embed -ef sms -cf "$variable" -sf secret"$contador".jpeg -p '1234'
fi
done
mv secret* imagenesconsms/
echo -e
echo -e

┌──(root㉿kali)-[/home/kali/Pictures]
└─#
```

Se crea script y se garantiza que haya guardado correctamente.

```
┌──(root㉿kali)-[/home/kali/Pictures]
└─# chmod +x srvestegano.py

┌──(root㉿kali)-[/home/kali/Pictures]
└─# ./srvestegano.py
embedding "sms" in "final.jpeg" ... done
writing stego file "secret1.jpeg" ... done
embedding "sms" in "Imagen1.jpeg" ... done
writing stego file "secret2.jpeg" ... done
embedding "sms" in "Imagen2.jpeg" ... done
writing stego file "secret3.jpeg" ... done
embedding "sms" in "Imagen3.jpeg" ... done
writing stego file "secret4.jpeg" ... done
embedding "sms" in "prueba.jpeg" ... done
writing stego file "secret5.jpeg" ... done



┌──(root㉿kali)-[/home/kali/Pictures]
└─#
```

Luego de ello, se dan permisos al archivo y se ejecuta el script.



Se confirma que se haya creado la carpeta imagenesconsms, y con un ls se confirma haya creado las imágenes nuevas.

¿Qué hace el script srvestegano.py?

El script busca imágenes en el directorio actual, oculta un mensaje SMS en cada una utilizando steghide, y luego mueve las imágenes con el mensaje oculto a un nuevo directorio llamado imagenesconsms.

¿Por qué es importante ocultar mensajes dentro de archivos de imagen?

Ocultar mensajes dentro de archivos de imagen puede ser importante por razones de seguridad, privacidad, protección de la propiedad intelectual y aplicaciones forenses. Sin embargo, es importante tener en cuenta que el uso de esteganografía también puede plantear preocupaciones éticas y legales, especialmente si se utiliza para actividades ilegales o maliciosas.

¿Cuáles son algunas aplicaciones prácticas de la esteganografía en la vida cotidiana?

La esteganografía tiene una variedad de aplicaciones prácticas en la vida cotidiana, incluida la protección de la privacidad, la seguridad de la información, la autenticación de documentos y la investigación forense. Su uso puede proporcionar una capa adicional de seguridad y protección en diversas situaciones.

¿Cuál es el riesgo de seguridad asociado con la esteganografía en la distribución de malware?

La esteganografía en la distribución de malware presenta importantes riesgos de seguridad al permitir que los atacantes evadan la detección, propaguen malware de manera sigilosa, realicen ataques dirigidos y exfiltran datos de manera encubierta.